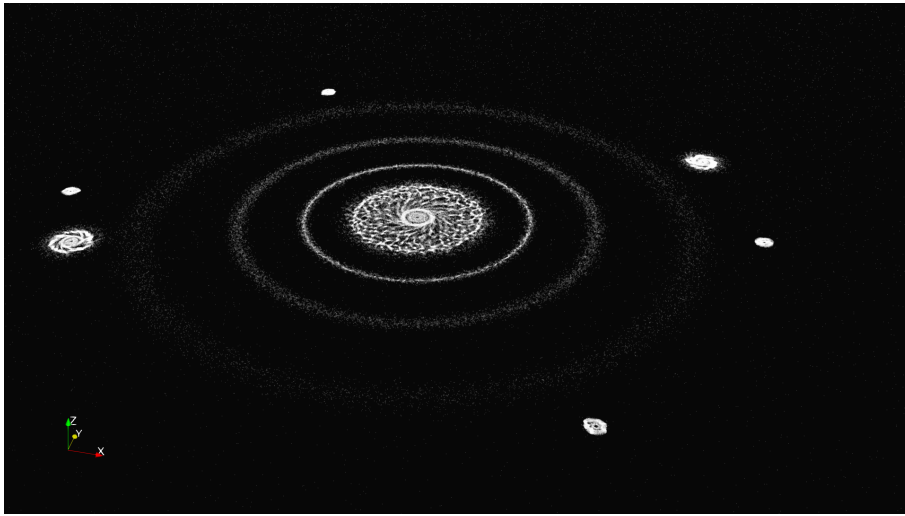


Simulazioni n -body su GPU: approccio naive e Barnes-Hut

Relazione del progetto di GPGPU - Damiano Trovato

Abstract

Con la seguente relazione si documenta il processo di realizzazione di una simulazione n -body tramite soluzioni parallele su GPU. Si trattano quindi aspetti degni di nota, quali gli algoritmi, i metodi numerici, le ottimizzazioni e un'analisi delle performance. Le soluzioni presentate saranno due: un'approccio naive su GPU, e un approccio approssimato Barnes-Hut, sempre su GPU.



1 Dettagli comuni alle soluzioni trattate

1.1 Aspetti matematici

Si offre una lista delle formule matematiche che rendono fattibili le simulazioni realizzate all'interno del progetto.

Legge di gravitazione universale La seguente formula detta l'attrazione che agisce tra due corpi rispettivamente di massa m_1 ed m_2 , situate a distanza R . Questa grandezza è direttamente proporzionale al prodotto delle masse e inversamente proporzionale al quadrato della distanza, tutto moltiplicato per una costante (di cui verrà modificato il valore ai fini della simulazione).

$$F = G \frac{m_1 m_2}{R^2} \quad (1)$$

In natura G è una costante minuscola che rende l'attrazione tra i corpi rilevante solo se i corpi hanno massa elevatissima o distanza reciproca ridotta. Ai fini della simulazione, questa costante verrà portata a valori molto più grandi.

Dalla forza all'accelerazione

$$F = ma \Rightarrow a = \frac{F}{m} \quad (2)$$

Dall'accelerazione alla velocità L'accelerazione è la derivata della velocità sul tempo, possiamo quindi ottenere

$$a = \frac{dv}{dt} \Rightarrow a dt = \frac{F}{m} dt = dv \quad (3)$$

fissando un valore di dt , tramite la variabile `delta_time`, andremo a integrare rispetto al tempo, ottenendo la velocità.

Dalla velocità allo spostamento

$$x(t) = x_0 + v dt \quad (4)$$

dove $dt = \text{delta_time}$.

1.2 Metodo numerico di integrazione: leapfrog

Il metodo numerico utilizzato nella simulazione, è noto come metodo **leapfrog**, o metodo della cavallina. Il nome deriva dal modo in cui si alterna l'integrazione della velocità a quella del tempo.

Pseudo-codice 1 Integrazione Leapfrog

- 1: Calcola la forza
 - 2: Aggiorna la velocità per $\Delta t/2$
 - 3: **for** Numero di iterazioni **do**
 - 4: Aggiorna la posizione per Δt
 - 5: Calcola la forza
 - 6: Aggiorna la velocità per Δt
 - 7: **end for**
-

Questo metodo numerico, sfasando di $\Delta t/2$ l'integrazione della velocità e della posizione, ottiene, nel contesto delle simulazioni n -body, risultati più accurati a parità di operazioni. L'accuratezza guadagnata, rispetto a un banale metodo di Eulero, si manifesta nell'energia del sistema, che viene conservata (con oscillazioni trascurabili).

2 Simulazione parallela naïve su GPU

Il primo approccio alla simulazione di sistemi a n corpi che si è deciso di realizzare, è una soluzione parallela basata sull'algoritmo naïve, con complessità computazionale $\mathcal{O}(n^2)$.

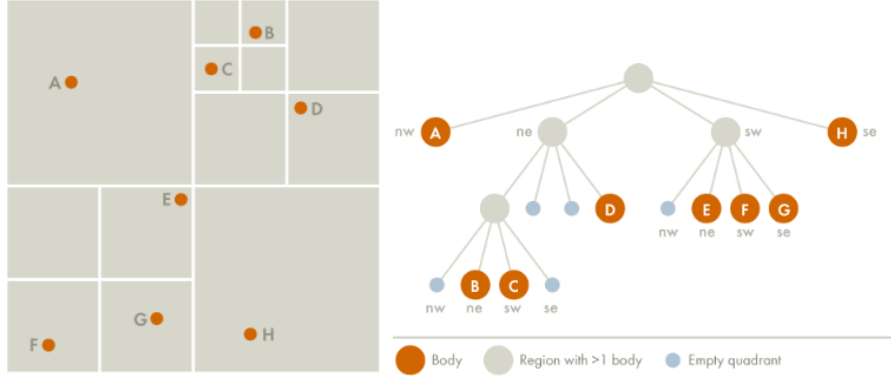
2.1 I kernel fondamentali

2.2 L'algoritmo complessivo

2.3 Valutazione delle performance

3 Simulazione approssimata Barnes-Hut su GPU

L'algoritmo Barnes-Hut permette di simulare sistemi a n corpi in maniera approssimata, con precisione arbitraria e complessità asintotica $\mathcal{O}(n \log n)$. Sfrutta una particolare struttura dati, ossia un quad-tree (figura 3) - un albero in cui ogni nodo ha quattro figli - per suddividere ricorsivamente lo spazio, memorizzare valori relativi al sistema e velocizzare il passaggio più oneroso della simulazione brute-force: il calcolo delle forze (o dell'accelerazione), notoriamente un passaggio a complessità $\mathcal{O}(n^2)$.



L'idea è la seguente: costruiamo un albero che descrive l'intero sistema, e invece di calcolare l'attrazione tra un corpo c e il resto dei corpi C , si visita l'albero, e per ogni altro nodo ci chiediamo "posso approssimare tutti i figli di questo nodo ad un unico corpo?". Lo **sviluppo in multipoli** ci dice che questo è possibile quando l'area che include questi corpi non è molto grande e quando la distanza da c è elevata: useremo un parametro θ soglia, che verrà confrontato proprio al rapporto tra le grandezze appena citate. Quando il criterio è rispettato, si accetta il nodo come approssimazione di tutti i corpi della regione associata, con posizione uguale al centro di massa, e massa uguale alla somma delle masse, altrimenti si continua la visita nell'albero. Su sistemi con molteplici regioni dense, ma molto distanti tra loro, l'algoritmo Barnes-Hut performa estremamente bene.

Tuttavia l'algoritmo paga l'overhead della costruzione del quad-tree, che richiede non pochi passaggi:

1. Definizione dell'area dell'intero sistema particellare, ossia la **bounding-box**.
2. Inserimento di ciascun corpo all'interno dell'albero.
3. Propagazione nei nodi interni delle masse e dei centri di massa dei rispettivi figli.

Questi passaggi sono i più costosi dell'algoritmo, ma che offrono proprio il vantaggio asintotico. Trattiamo adesso come è stato possibile ottenere un vantaggio tramite la programmazione parallela su GPU.

3.1 I kernel fondamentali

Calcolo dei margini della bounding-box I primi kernel sono quelli dedicati alla definizione della dimensione della bounding-box (ossia della radice del quad-tree). Usiamo una riduzione parallela per trovare il minimo e il massimo dell'intero sistema particellare, di ciascuna delle due dimensioni. In questo modo, sarà possibile costruire un cubo che contiene l'intero sistema, per ciascun istante di tempo t .

Pseudo-codice 2 Kernel per la riduzione parallela (ricerca del minimo o massimo)

Require: array A , array B , dimensione dell'input n , array $BoundingBox$, flag per l'ultima riduzione

```

1:  $j \leftarrow \text{id globale del work-item}$ 
2:  $i \leftarrow 2j$ 
3: if  $i > n$  then return            $\triangleright$  Usciamo dal programma se il work-item è fuori dal dominio dell'input
4: end if
5:  $val \leftarrow A[i]$ 
6: if  $i + 1 < n$  then
7:    $val \leftarrow \min(val, A[i + 1])$             $\triangleright \max(val, A[i + 1])$  nella ricerca del massimo
8: end if
9: if è l'ultima riduzione then
10:   $BoundingBox[\text{MIN\_INDEX}] \leftarrow val$             $\triangleright BoundingBox[\text{MAX\_INDEX}]$  nella ricerca del
11: end if
12:  $B[j] \leftarrow val$ 

```

Questo kernel sarà lanciato fino a quando il minimo (massimo) si troverà alla posizione 0 dell'ultimo array usato. Nel codice host, abbiamo deciso di usare due array, scambiando ogni volta l'array di input e quello d'output (una sorta di ping-pong tra gli array).

Costruzione del quad-tree Il secondo kernel che presentiamo, è quello relativo alla **costruzione del quad-tree**. Prima di presentare il kernel, definiamo in cosa consiste il nostro quad-tree in termini di strutture dati. Necessitiamo infatti un modo per linearizzare un quad-tree in un array (sia per le performance, che per le capacità)

```
int cell_children = array dei figli di ciascuna cella. 4 volte più lungo degli altri;
float2 cell_center = array delle posizioni (x,y) dei centri delle celle;
float cell_half_size = array di metà lunghezza del lato delle celle;
float2 cell_center_of_mass_mem = centro di massa della cella (basato sui figli);
float cell_mass_mem = massa della cella (somma delle masse dei figli);
```

La gestione degli indici in relazione ai figli è banale: in questo blog [?] viene fornito un esempio banale:

```

/-----|-----|
|0| |1|2|3|4| |5|6|7|8|9|10|11|12|13|...
\__|_____|
```

Nel nostro caso, nell'array dedicato ai figli, i 4 figli della radice si avranno in posizione 0,1,2,3. La profondità di un indice i nell'albero può inoltre essere determinata da $\lceil \log_4(i) \rceil + 1$. A parità di profondità, l'ordinamento dei quattro nodi rimane a nostra discrezione. Stabiliamo la seguente suddivisione, rispetto alla suddivisione dei quadranti:

```

/* CONVENZIONE:
   10 11
   00 01
*/
inline int get_quadrant(float2 pos, float2 center) {
    int right = pos.x >= center.x;
    int top = pos.y >= center.y;
    return (top << 1) | right;
}
```

In questo modo, accedere ai quattro figli di un nodo i consisterà nel calcolo $4i + j$, con $j = 0$ per il figlio che indica il quadrante in basso a sinistra, $j = 1$ per il quadrante in basso a destra, $j = 2$ per il quadrante in alto a sinistra e $j = 3$ per il quadrante in alto a destra.

In letteratura non si trovano esempi di algoritmi di costruzione di quad-tree paralleli su GPU senza ausilio delle funzioni atomiche. Usiamo quindi un approccio seriale, mantenendo tuttavia i dati su GPU, per risparmiare sullo scambio dati CPU-GPU. Ci rifaremo quindi alla costruzione del quad-tree dell'algoritmo originale, ossia.

1. Se il nodo corrente c non contiene un corpo, inseriamo il corpo b in c .
2. Se il nodo c è un nodo interno, inseriamo il corpo b

3.2 L'algoritmo complessivo

3.3 Valutazione delle performance

